

Adding a Sensor

This guide explains how GeoSenEsm sensor integrations are configured and where the boundary is between a configuration-only change and a mobile application rebuild.

GeoSenEsm stores sensor readings in a dynamic shape: one reading envelope plus named parameter values. The same `sensor_type.code` and `sensor_parameter_definition.code` values must flow through the backend, admin panel, mobile app, SQL storage, and Mongo response documents.

What Can Be Changed Without Rebuilding the Mobile App

You can add or revise a sensor without rebuilding the mobile application when the sensor fits the existing profile engine:

1. The sensor communicates over Bluetooth Low Energy.
2. The phone can find it by advertised name, name prefix, service UUID, manufacturer data, or Xiaomi MiBeacon service data.
3. The read sequence can be described as `read`, `notification`, `indication`, `write`, and `delay` steps.
4. The payload can be decoded using the supported scalar types: `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `float32`, or `sfloat16`.
5. The frame validation only needs a fixed length, optional prefix, no checksum, or `crc8_maxim`.
6. The output values can be represented as configured sensor parameters such as `temperature`, `humidity`, `spo2`, `pulse_rate`, `light`, `moisture`, `conductivity`, `opening`, or another active parameter code.

In this case the deployment work is data/configuration:

1. Add the `sensor_type` row with `integration_mode = 'profile'`.
2. Add or enable the `sensor_type_setting` row and choose the connection timeout.
3. Add any missing `sensor_parameter_definition` rows.
4. Map parameters to the source in `sensor_parameter_source`.
5. Create and publish a `sensor_gatt_profile` revision.
6. Assign the source or physical device to respondents.
7. Ask respondents to refresh the mobile app so `/api/surveysettings/sensordata/mobile` syncs the new setup.

The app rebuild is not needed because the backend sends the active profile to the existing BLE profile engine.

When a Rebuild Is Still Required

Rebuild and release the mobile app when the new sensor needs behavior outside the profile engine:

1. A new transport such as Classic Bluetooth, USB, NFC, Wi-Fi, or a vendor SDK.
2. A proprietary handshake that cannot be expressed as profile `write` and `delay` steps.
3. A new encrypted advertisement decoder. The current advertisement decoder whitelist supports Xiaomi MiBeacon v4/v5.
4. A new checksum, compression, multi-packet state machine, or data type not supported by the profile validator.
5. New mobile permissions, background behavior, or user interface changes.
6. A native/coded adapter registered in the mobile app instead of `integration_mode = 'profile'`.

Profile Shape

A published profile is stored in `sensor_gatt_profile.spec_json`. It describes how to discover the device, which BLE operations to perform, how to validate returned frames, and how bytes become parameter values.

Minimal GATT profile:

```
{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "nameExact": "Device name",
    "serviceUuid": "00000000-0000-1000-8000-00805f9b34fb"
  },
  "operations": [
    {
      "kind": "acquire",
      "serviceUuid": "00000000-0000-1000-8000-00805f9b34fb",
      "characteristicUuid": "00000001-0000-1000-8000-00805f9b34fb",
      "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
      "frame": { "length": 3, "prefixHex": "", "checksum": "none" },
      "assertions": [],
      "decoders": [
        {
          "parameter": "temperature",
          "type": "uint16",
          "offset": 0,
          "endian": "little",
          "scale": 0.01,
          "add": 0,

```


Xiaomi LYWSD03MMC Temperature and Humidity

Xiaomi LYWSD03MMC is a simple read-based GATT profile. The app discovers the device by name and service UUID, reads one characteristic, and decodes temperature and humidity from three bytes.

```
{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "nameExact": "LYWSD03MMC",
    "serviceUuid": "ebe0ccb0-7a0a-4b0c-8a1a-6ff2997da3a6"
  },
  "operations": [
    {
      "kind": "acquire",
      "serviceUuid": "ebe0ccb0-7a0a-4b0c-8a1a-6ff2997da3a6",
      "characteristicUuid": "ebe0ccc1-7a0a-4b0c-8a1a-6ff2997da3a6",
      "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
      "frame": { "length": 3, "prefixHex": "", "checksum": "none" },
      "assertions": [],
      "decoders": [
        { "parameter": "temperature", "type": "uint16", "offset": 0, "endian": "little", "scale": 100 },
        { "parameter": "humidity", "type": "uint8", "offset": 2, "endian": "little", "scale": 100 }
      ]
    }
  ],
  "goldenPackets": [
    {
      "characteristicUuid": "ebe0ccc1-7a0a-4b0c-8a1a-6ff2997da3a6",
      "packetHex": "340C2D",
      "expected": { "temperature": 31.24, "humidity": 45 }
    }
  ],
  "requiredSecrets": []
}
```

Kestrel Drop 2 Temperature and Humidity

Kestrel Drop 2 exposes temperature and humidity as two separate readable characteristics. The profile can discover a respondent-assigned device by interpolating the physical sensor identifier into D2 - {sensorId}, while also accepting the shared Kestrel service UUID.

```
{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
```

```

"discovery": {
  "nameExact": "D2 - {sensorId}",
  "namePrefix": "D2 - ",
  "serviceUuid": "12630000-cc25-497d-9854-9b6c02c77054"
},
"operations": [
  {
    "kind": "acquire",
    "serviceUuid": "12630000-cc25-497d-9854-9b6c02c77054",
    "characteristicUuid": "12630001-cc25-497d-9854-9b6c02c77054",
    "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
    "frame": { "length": 3, "prefixHex": "", "checksum": "none" },
    "assertions": [{ "offset": 0, "equals": 7 }],
    "decoders": [
      { "parameter": "temperature", "type": "uint16", "offset": 1, "endian": "little", "scale": 1 }
    ]
  },
  {
    "kind": "acquire",
    "serviceUuid": "12630000-cc25-497d-9854-9b6c02c77054",
    "characteristicUuid": "12630002-cc25-497d-9854-9b6c02c77054",
    "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
    "frame": { "length": 3, "prefixHex": "", "checksum": "none" },
    "assertions": [{ "offset": 0, "equals": 7 }],
    "decoders": [
      { "parameter": "humidity", "type": "uint16", "offset": 1, "endian": "little", "scale": 1 }
    ]
  }
],
"goldenPackets": [
  {
    "characteristicUuid": "12630001-cc25-497d-9854-9b6c02c77054",
    "packetHex": "073408",
    "expected": { "temperature": 21 }
  },
  {
    "characteristicUuid": "12630002-cc25-497d-9854-9b6c02c77054",
    "packetHex": "079411",
    "expected": { "humidity": 45 }
  }
],
"requiredSecrets": []
}

```

PC-60FW Pulse Oximeter

PC-60FW uses notifications. The profile validates the frame prefix and crc8_maxim checksum, then maps packet bytes to SpO2, pulse rate, and perfusion index.

```
{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "nameExact": "PC-60FW",
    "serviceUuid": "6e400001-b5a3-f393-e0a9-e50e24dcca9e"
  },
  "operations": [
    {
      "kind": "acquire",
      "serviceUuid": "6e400001-b5a3-f393-e0a9-e50e24dcca9e",
      "characteristicUuid": "6e400003-b5a3-f393-e0a9-e50e24dcca9e",
      "acquisition": { "mode": "notification", "timeoutMs": 30000, "maxPackets": 10 },
      "frame": { "length": 12, "prefixHex": "AA550F0801", "checksum": "crc8_maxim" },
      "assertions": [],
      "decoders": [
        { "parameter": "spo2", "type": "uint8", "offset": 5, "endian": "little", "scale": 1 },
        { "parameter": "pulse_rate", "type": "uint8", "offset": 6, "endian": "little", "scale": 1 },
        { "parameter": "perfusion_index", "type": "uint8", "offset": 8, "endian": "little", "scale": 1 }
      ]
    }
  ],
  "goldenPackets": [
    {
      "characteristicUuid": "6e400003-b5a3-f393-e0a9-e50e24dcca9e",
      "packetHex": "AA550F08016248000F000079",
      "expected": { "spo2": 98, "pulse_rate": 72, "perfusion_index": 1.5 }
    }
  ],
  "requiredSecrets": []
}
```

Bluetooth SIG PLX Pulse Oximeter

Bluetooth SIG PLX discovers devices by the standard pulse oximeter service UUID. The profile listens for notifications and decodes the standard SFLOAT fields.

```
{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
```

```

"discovery": {
  "serviceUuid": "00001822-0000-1000-8000-00805f9b34fb"
},
"operations": [
  {
    "kind": "acquire",
    "serviceUuid": "00001822-0000-1000-8000-00805f9b34fb",
    "characteristicUuid": "00002a5f-0000-1000-8000-00805f9b34fb",
    "acquisition": { "mode": "notification", "timeoutMs": 30000, "maxPackets": 10 },
    "frame": { "length": 5, "prefixHex": "", "checksum": "none" },
    "assertions": [],
    "decoders": [
      { "parameter": "flags", "type": "uint8", "offset": 0, "endian": "little", "scale": 1 },
      { "parameter": "spo2", "type": "sfloat16", "offset": 1, "endian": "little", "scale": 1 },
      { "parameter": "pulse_rate", "type": "sfloat16", "offset": 3, "endian": "little", "scale": 1 }
    ]
  }
],
"goldenPackets": [
  {
    "characteristicUuid": "00002a5f-0000-1000-8000-00805f9b34fb",
    "packetHex": "0062004800",
    "expected": { "flags": 0, "spo2": 98, "pulse_rate": 72 }
  }
],
"requiredSecrets": []
}

```

Flower Care Plant Sensor

Flower Care shows a multi-step sequence. The app writes A01F, waits for the device to prepare data, then reads one characteristic and maps four values.

```

{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "nameExact": "Flower care",
    "serviceUuid": "00001204-0000-1000-8000-00805f9b34fb"
  },
  "operations": [
    {
      "kind": "write",
      "serviceUuid": "00001204-0000-1000-8000-00805f9b34fb",
      "characteristicUuid": "00001a00-0000-1000-8000-00805f9b34fb",
      "payloadHex": "A01F",

```

```

        "timeoutMs": 5000
    },
    { "kind": "delay", "durationMs": 750 },
    {
        "kind": "acquire",
        "serviceUuid": "00001204-0000-1000-8000-00805f9b34fb",
        "characteristicUuid": "00001a01-0000-1000-8000-00805f9b34fb",
        "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
        "frame": { "length": 10, "prefixHex": "", "checksum": "none" },
        "assertions": [],
        "decoders": [
            { "parameter": "temperature", "type": "int16", "offset": 0, "endian": "little", "scale": 1 },
            { "parameter": "light", "type": "uint32", "offset": 3, "endian": "little", "scale": 1 },
            { "parameter": "moisture", "type": "uint8", "offset": 7, "endian": "little", "scale": 1 },
            { "parameter": "conductivity", "type": "uint16", "offset": 8, "endian": "little", "scale": 1 }
        ]
    }
],
"goldenPackets": [
    {
        "characteristicUuid": "00001a01-0000-1000-8000-00805f9b34fb",
        "packetHex": "D70000393000002D5E01",
        "expected": { "temperature": 21.5, "light": 12345, "moisture": 45, "conductivity": 350 }
    }
],
"requiredSecrets": []
}

```

Implementation Checklist

Use the configuration-only path when possible:

1. Create the sensor type with a stable lowercase code and `integration_mode = 'profile'`.
2. Define active parameter codes in `sensor_parameter_definition`.
3. Link the parameter codes to the sensor type in `sensor_parameter_source`.
4. Create a draft profile, validate it, and publish it.
5. Enable the source and set its timeout in Survey Settings.
6. Assign the source or physical device to respondents.
7. Refresh the mobile app and complete a test survey.
8. Confirm SQL stores one `sensor_data` row with matching `sensor_data_parameter_value` rows.
9. Confirm response document JSON contains `sensorData.source` and `sensorData.values`.

Use the code-change path only when the sensor cannot be represented by the profile engine:

1. Backend: extend the model or validation only if the submitted data shape or profile schema must change.
2. Admin panel: update settings screens only when a new persisted option or visible concept is needed.
3. Mobile app: add a coded adapter, parser, permission, or UI only when the profile engine cannot perform the integration.
4. Add tests at the layer that changed.